

Module 4: Node.js, Express & TypeORM › Section 13: Sessions, Authentication & Security › 13-03
Security Practices

Security Practices: Helmet, CORS, Rate Limiting & Input Escaping

SECTION INFO

Section: 13 | Subtopic: 03 | Estimated Duration: 2–3 hours

Theory

Practical

Security

Topics covered: HTTP security headers with Helmet, Cross-Origin Resource Sharing (CORS), request rate limiting with `express-rate-limit`, HTML/SQL input escaping and sanitisation

Learning Objectives — This Subtopic

- Configure Helmet to set defensive HTTP response headers and understand what each header protects against
- Implement CORS middleware with fine-grained origin and method policies appropriate for both development and production
- Apply `express-rate-limit` to protect endpoints from brute-force and denial-of-service abuse
- Distinguish between input validation (covered in Section 8) and input escaping/sanitisation, and know when each is required
- Integrate all three layers (headers, rate limiting, escaping) into a single cohesive Express + TypeORM application
- Reason about the threat model behind each control so decisions are deliberate rather than cargo-culted

1. Why Application-Level Security Controls Matter

Database authentication and HTTPS at the transport layer protect data in transit and at rest. They do not protect against a browser loading your API response into a malicious page, an attacker enumerating credentials with a script, or a stored cross-site scripting (XSS) payload that executes when an admin views a dashboard. Application-layer controls fill those gaps.

The controls this document covers operate at three distinct points in the request lifecycle:

- **Before the request is processed** — rate limiting drops abusive traffic early.
- **During response construction** — output escaping prevents injected content from being interpreted as code.
- **In the response headers** — Helmet and CORS headers instruct the browser how to handle the response.

Analogy: Think of a restaurant kitchen. The bouncer at the door (rate limiting) stops obviously disruptive guests. The chef sanitises ingredients before cooking (input escaping). The health inspector's certificate on the wall (security headers) signals to diners that standards are met. Removing any one layer does not make the others pointless, but every removed layer is a gap an attacker can walk through.

2. Project Setup

All examples in this document build on a minimal TypeScript Express + TypeORM project. Install the required packages first.

```
npm install express helmet cors express-rate-limit he
```

```
npm install --save-dev typescript @types/node @types/express @types/cors @types/he ts-node-dev
```

The `he` package is a robust HTML entity encoder/decoder used for output escaping later in this document.

```
project-root/ |— package.json |— tsconfig.json |— src/ |— app.ts # Express app factory
(middleware registration) |— server.ts # HTTP server entry point |— middleware/ | |—
rateLimiter.ts | |— sanitise.ts |— routes/ |— comments.ts # Example route used in practicals
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "commonjs",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  }
}
```

3. Helmet — HTTP Security Headers

3.1 What HTTP Headers Have to Do With Security

Browsers implement a large number of security features that are opt-in: a server must explicitly declare that it wants them. These opt-ins are expressed as HTTP response headers. Without them, the browser falls back to permissive legacy behaviour that existed before these features were standardised.

Helmet is a collection of small Express middleware functions, each responsible for one header. Calling `helmet()` with no arguments applies a sensible default set. You can override individual directives when the defaults are too restrictive for your application.

3.2 Stage 1 — Applying the Default Helmet Configuration

STAGE 1

src/app.ts

```
import express, { Application } from 'express';
import helmet from 'helmet';

const app: Application = express();

app.use(helmet());

app.get('/ping', (_req, res) => {
  res.json({ message: 'pong' });
});

export default app;
```

src/server.ts

```
import app from './app';

const PORT = parseInt(process.env.PORT ?? '3000', 10);

app.listen(PORT, () => {
  console.log(`Server listening on port ${PORT}`);
});
```

`npx ts-node-dev src/server.ts`

Inspect the response headers in a separate terminal:

`curl -I http://localhost:3000/ping`

Output:

```
HTTP/1.1 200 OK
Content-Security-Policy: default-src 'self';base-uri 'self';font-src 'self' https: data:;...
Cross-Origin-Opener-Policy: same-origin
Cross-Origin-Resource-Policy: same-origin
Origin-Agent-Cluster: ?1
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=15552000; includeSubDomains
X-Content-Type-Options: nosniff
X-DNS-Prefetch-Control: off
X-Download-Options: noopen
X-Frame-Options: SAMEORIGIN
```

```
X-Permitted-Cross-Domain-Policies: none
X-XSS-Protection: 0
Content-Type: application/json; charset=utf-8
```

Without Helmet, none of those green headers would be present. Each one instructs the browser to apply a specific defence.

3.3 What Each Default Header Does

Header	Threat Mitigated	What the Default Does
<code>Content-Security-Policy</code> (CSP)	XSS, data injection	Restricts which sources can load scripts, styles, fonts, frames
<code>X-Frame-Options</code>	Clickjacking	<code>SAMEORIGIN</code> — page may only be framed by itself
<code>X-Content-Type-Options</code>	MIME sniffing	<code>nosniff</code> — browser must honour declared <code>Content-Type</code>
<code>Strict-Transport-Security</code> (HSTS)	SSL stripping, downgrade attacks	Forces HTTPS for 180 days after first visit
<code>Referrer-Policy</code>	Information leakage via <code>Referer</code> header	<code>no-referrer</code> — suppresses referrer entirely
<code>Cross-Origin-Opener-Policy</code>	Cross-origin isolation bypass	<code>same-origin</code> — prevents shared browsing context with other origins
<code>X-XSS-Protection</code>	Legacy IE XSS filter (now harmful)	Set to <code>0</code> — disables the buggy legacy filter; CSP is the modern replacement

Key Insight: `X-XSS-Protection: 0` may look counterintuitive. The old IE XSS filter had exploitable bugs — disabling it and relying on CSP instead is the current recommendation from major browser vendors and the OWASP Cheat Sheet.

3.4 Stage 2 — Customising the Content Security Policy

The default CSP blocks all inline scripts, which is correct for pure API servers. If your application serves a frontend that loads scripts from a CDN or uses inline `<style>` tags, you need a tailored directive.

STAGE 2

`src/app.ts`

```
import express, { Application } from 'express';
import helmet from 'helmet';

const app: Application = express();

app.use(
  helmet({
    contentSecurityPolicy: {
      directives: {
        defaultSrc: ["'self'"],
        scriptSrc: ["'self'", 'https://cdn.jsdelivr.net'],
        styleSrc: ["'self'", "'unsafe-inline'"],
        imgSrc: ["'self'", 'data:', 'https:'],
        connectSrc: ["'self'"],
        frameAncestors: ["'none'"], // Stricter than SAMEORIGIN
        upgradeInsecureRequests: [], // Rewrite http:// sub-resources to https://
      },
    },
    // HSTS is only meaningful over HTTPS – disable in local dev if needed
    strictTransportSecurity: process.env.NODE_ENV === 'production'
      ? { maxAge: 31536000, includeSubDomains: true, preload: true }
      : false,
  })
);

app.get('/ping', (_req, res) => {
  res.json({ message: 'pong' });
});

export default app;
```

Output:

```
Content-Security-Policy: default-src 'self';script-src 'self' https://cdn.jsdelivr.net;style-src 'self'
'unsafe-inline';img-src 'self' data: https;;connect-src 'self';frame-ancestors 'none';upgrade-insecure-requests
Strict-Transport-Security: (omitted in development – NODE_ENV not set to production)
```

The key change is that `frameAncestors: ["'none'"]` is set in CSP rather than relying solely on `X-Frame-Options`. Modern browsers honour the CSP directive; `X-Frame-Options` is retained for older browsers that do not understand CSP Level 2.

Warning: Never place `'unsafe-inline'` or `'unsafe-eval'` in `scriptSrc` in production. These directives negate most of CSP's XSS protection. If a third-party library requires them, find a nonce-based or hash-based alternative instead.

4. CORS — Cross-Origin Resource Sharing

4.1 Why the Same-Origin Policy Exists and What CORS Does

Browsers enforce the same-origin policy: a page loaded from `https://app.example.com` cannot make AJAX requests to `https://api.example.com` unless the server at `api.example.com` explicitly permits it. This prevents a malicious page from silently reading your banking data using your session cookie.

CORS (Cross-Origin Resource Sharing) is the standard mechanism by which a server signals its cross-origin policy. The browser performs a **preflight** OPTIONS request before any non-simple cross-origin request; if the server's CORS headers do not match, the browser blocks the response — the request never reaches your route handler.

Key Insight: CORS is enforced by the *browser*, not the server. It is not a server-side firewall. A non-browser HTTP client (curl, Postman, another server) ignores CORS headers entirely. CORS protects users, not your API endpoints from direct access — rate limiting and authentication guard those.

4.2 Stage 1 — Permissive CORS for Local Development

STAGE 1

`src/app.ts`

```
import express, { Application } from 'express';
import helmet from 'helmet';
import cors from 'cors';

const app: Application = express();

app.use(helmet());
app.use(cors()); // Allows all origins – development only

app.get('/ping', (_req, res) => {
  res.json({ message: 'pong' });
});

export default app;
```

```
curl -I -H "Origin: http://localhost:4200" http://localhost:3000/ping
```

Output:

```
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
```

The wildcard `*` means any origin may read the response. This is acceptable for fully public, unauthenticated API data, but a critical security flaw if your API sets cookies or requires credentials — browsers refuse to send credentials to wildcard-CORS endpoints.

4.3 Stage 2 — Allowlist-Based CORS for Production

STAGE 2

`src/app.ts`

```
import express, { Application, Request, Response, NextFunction } from 'express';
import helmet from 'helmet';
import cors, { CorsOptions } from 'cors';

const ALLOWED_ORIGINS: string[] = (
  process.env.ALLOWED_ORIGINS ?? 'http://localhost:4200'
```



```

).split(',').map(o => o.trim());

const corsOptions: CorsOptions = {
  origin: (origin, callback) => {
    // Allow server-to-server requests (no Origin header) or listed origins
    if (!origin || ALLOWED_ORIGINS.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error(`CORS policy does not allow origin: ${origin}`));
    }
  },
  methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true,           // Allow cookies / Authorization header
  maxAge: 86400,              // Cache preflight result for 24 hours
};

const app: Application = express();

app.use(helmet());
app.use(cors(corsOptions));
app.options('*', cors(corsOptions)); // Handle preflight for all routes

app.use(express.json());

app.get('/ping', (_req, res) => {
  res.json({ message: 'pong' });
});

// Generic error handler – catches CORS rejections among others
app.use((err: Error, _req: Request, res: Response, _next: NextFunction) => {
  res.status(403).json({ error: err.message });
});

export default app;
```

Test with a permitted origin and a rejected one:

```
curl -I -H "Origin: http://localhost:4200" http://localhost:3000/ping
```

Output:

```
Access-Control-Allow-Origin: http://localhost:4200
Access-Control-Allow-Credentials: true
Vary: Origin
```

```
curl -I -H "Origin: http://evil.example.com" http://localhost:3000/ping
```

Output:

```
HTTP/1.1 403 Forbidden
{"error":"CORS policy does not allow origin: http://evil.example.com"}
```

Notice the `Vary: Origin` header. Because the response differs per origin, CDNs must be instructed not to serve one user's response to another — the `cors` package adds this automatically when origin matching is dynamic.

4.4 Applying CORS to Specific Route Groups

When your API serves both a public read-only segment and a private write segment, you can apply different CORS policies per router rather than globally.

`src/app.ts` (excerpt)

```
import publicRouter from './routes/public';
import adminRouter from './routes/admin';

const publicCors = cors({ origin: '*' });
const restrictedCors = cors(corsOptions);

app.use('/api/public', publicCors, publicRouter);
app.use('/api/admin', restrictedCors, adminRouter);
```

Output:

```
GET /api/public/products → Access-Control-Allow-Origin: *
GET /api/admin/users → Access-Control-Allow-Origin: https://dashboard.example.com (if allowed)
```

5. Rate Limiting with `express-rate-limit`

5.1 What Rate Limiting Defends Against

Without rate limiting, an attacker can:

- Enumerate valid usernames via a login endpoint (credential stuffing)
- Brute-force a PIN or OTP reset endpoint
- Flood your database with write requests until it becomes unavailable
- Amplify costs on pay-per-request services (e.g., SMS verification, AI API calls)

Rate limiting counts requests from a given client over a time window and returns HTTP `429 Too Many Requests` once the threshold is exceeded. It is not a complete defence against a distributed botnet, but it raises the cost of an attack significantly and is trivial to implement.

5.2 Stage 1 — Global Rate Limiter

STAGE 1

`src/middleware/rateLimiter.ts`

```
import rateLimit from 'express-rate-limit';

export const globalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15-minute window
  max: 100,                 // Max requests per IP per window
  standardHeaders: true,    // Return rate limit info in RateLimit-* headers (RFC 6585)
  legacyHeaders: false,    // Disable deprecated X-RateLimit-* headers
  message: {
    status: 429,
    error: 'Too many requests, please try again later.',
  },
});
```

`src/app.ts`

```
import express, { Application } from 'express';
import helmet from 'helmet';
import cors from 'cors';
import { globalLimiter } from '../middleware/rateLimiter';
```

```
const app: Application = express();

app.use(helmet());
app.use(cors());
app.use(globalLimiter); // Applied before any route

app.use(express.json());

app.get('/ping', (_req, res) => {
  res.json({ message: 'pong' });
});

export default app;
```

The `RateLimit-*` headers tell the client how many requests remain and when the window resets:

```
curl -I http://localhost:3000/ping
```

Output:

```
RateLimit-Limit: 100
RateLimit-Remaining: 99
RateLimit-Reset: 2026-03-01T12:15:00.000Z
```

5.3 Stage 2 — Endpoint-Specific Limiter for Auth Routes

A global limiter of 100 requests per 15 minutes is too permissive for a login endpoint — 100 password attempts is a trivial brute-force. Apply a tighter limiter to sensitive routes.

STAGE 2

`src/middleware/rateLimiter.ts`

```
import ratelimit from 'express-rate-limit';

export const globalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
  standardHeaders: true,
  legacyHeaders: false,
  message: { status: 429, error: 'Too many requests, please try again later.' },
});

export const authLimiter = rateLimit({
```

```
windowMs: 15 * 60 * 1000,
max: 10,           // Only 10 login attempts per 15 minutes
standardHeaders: true,
legacyHeaders: false,
skipSuccessfulRequests: true, // A successful login does not count against the limit
message: {
  status: 429,
  error: 'Too many login attempts. Please wait before trying again.',
},
});

export const passwordResetLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, // 1-hour window
  max: 3,                   // 3 reset emails per hour per IP
  standardHeaders: true,
  legacyHeaders: false,
  message: {
    status: 429,
    error: 'Password reset limit reached. Try again in an hour.',
  },
});
```

src/routes/auth.ts

```
import { Router, Request, Response } from 'express';
import { authLimiter, passwordResetLimiter } from '../middleware/rateLimiter';

const router = Router();

router.post('/login', authLimiter, (req: Request, res: Response) => {
  // Section 13-02 covered JWT-based authentication logic
  res.json({ message: 'Login endpoint – limited to 10 attempts per 15 min' });
});

router.post('/password-reset', passwordResetLimiter, (req: Request, res: Response) => {
  res.json({ message: 'Password reset endpoint – limited to 3 per hour' });
});

export default router;
```

Output:

```
POST /auth/login (1st-10th attempt) → 200 OK | RateLimit-Remaining: 9 ... 0
POST /auth/login (11th attempt)      → 429 Too Many Requests
                                     {"status":429,"error":"Too many login attempts. Please wait before
trying again."}
```

5.4 Stage 3 — Using a Shared Store for Multi-Process Deployments

The default `express-rate-limit` store is in-memory. If your application runs on multiple Node.js processes or instances (e.g., via PM2 cluster mode or a container replica), each process maintains its own counter. An attacker could send 10 requests to each of four replicas — 40 total — while appearing to be within the limit on every individual instance.

For production, use a shared store backed by Redis.

STAGE 3 `npm install rate-limit-redis ioredis` `npm install --save-dev @types/ioredis`

`src/middleware/rateLimiter.ts` (production-ready)

```
import rateLimit from 'express-rate-limit';
import RedisStore from 'rate-limit-redis';
import Redis      from 'ioredis';

const redis = new Redis({
  host: process.env.REDIS_HOST ?? '127.0.0.1',
  port: parseInt(process.env.REDIS_PORT ?? '6379', 10),
});

const makeStore = (prefix: string) =>
  new RedisStore({
    sendCommand: (...args: string[]) => redis.call(...args) as any,
    prefix,
  });

export const globalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100,
  standardHeaders: true,
  legacyHeaders: false,
  store: makeStore('rl:global:'),
  message: { status: 429, error: 'Too many requests.' },
});

export const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
```

```
max: 10,
standardHeaders: true,
legacyHeaders: false,
skipSuccessfulRequests: true,
store: makeStore('rl:auth:'),
message: { status: 429, error: 'Too many login attempts.' },
});
```

Note: When Redis is unavailable, `rate-limit-redis` throws rather than silently falling back to in-memory. Handle Redis connection errors in your application startup code and decide whether a failed Redis connection should prevent the server from accepting traffic.

6. Input Escaping and Output Sanitisation

6.1 Validation vs. Escaping — Two Different Problems

Section 8 of this module covered input *validation*: rejecting data that does not conform to an expected shape (wrong type, length, format). Escaping is a complementary concern that addresses what happens when valid data is subsequently rendered or interpreted in a different context.

Concern	Question Asked	Action on Failure	Library
Validation	Is this value the right type/shape?	Reject (400 Bad Request)	<code>joi</code> , <code>class-validator</code>
Escaping	Is this value safe to render in an HTML/SQL/shell context?	Encode special characters before rendering	<code>he</code> , <code>TypeORM</code> parameterised queries
Sanitisation	Should certain allowed HTML tags/attributes be stripped?	Remove or whitelist markup before storage/rendering	<code>dompurify</code> , <code>sanitize-html</code>

6.2 XSS via Stored User Content

Cross-site scripting (XSS) occurs when user-supplied text is rendered in a browser context where it can be interpreted as HTML or JavaScript. A stored XSS vulnerability allows an attacker to persist a payload in your database and have it execute in every victim's browser.

Consider a comment stored in a TypeORM `Comment` entity and later rendered into an HTML response. If the comment body is inserted directly into the HTML without escaping, any `<script>` tag in the text executes.

`src/middleware/sanitise.ts`

```
import he from 'he';

/**
 * Escapes all HTML special characters in a string.
 * Safe for use in text nodes and attribute values (not inside event attributes).
 */
export function escapeHtml(raw: string): string {
  return he.encode(raw, { useNamedReferences: false });
}

/**
 * Trims whitespace and escapes HTML in a single pass.
 * Use when storing or displaying free-text user input.
 */
export function sanitiseUserText(raw: unknown): string {
  if (typeof raw !== 'string') {
    throw new TypeError('Expected a string value');
  }
  return escapeHtml(raw.trim());
}
```

STAGE 1

First, see the problem without escaping:

`src/routes/comments.ts (unsafe – do not ship)`

```
import { Router, Request, Response } from 'express';

const router = Router();
```



```
// Simulated stored comment – imagine this came from the database
const storedComment = '<script>alert("XSS!")</script> Great article!';

router.get('/unsafe', (_req: Request, res: Response) => {
  // Directly interpolating user content into HTML – dangerous
  res.send(`<html><body><p>${storedComment}</p></body></html>`);
});

export default router;
```

Output:

Browser renders: <html><body><p>
→ <script> tag executes: alert("XSS!") fires immediately

STAGE 2

Apply `escapeHtml` before interpolation:

src/routes/comments.ts (safe)

```
import { Router, Request, Response } from 'express';
import { escapeHtml } from '../middleware/sanitise';

const router = Router();

const storedComment = '<script>alert("XSS!")</script> Great article!';

router.get('/safe', (_req: Request, res: Response) => {
  const safe = escapeHtml(storedComment);
  res.send(`<html><body><p>${safe}</p></body></html>`);
});

export default router;
```

Output:

Browser renders visible text (no script execution):
<script>alert("XSS!")</script> Great article!

The `he.encode()` function converts `<` to `<`, `>` to `>`, `"` to `"`, and `&` to `&`. The browser treats the encoded characters as literal text content, not HTML markup.

6.3 SQL Injection and TypeORM Parameterised Queries

TypeORM's repository methods and `QueryBuilder` use parameterised queries by default — user input never becomes part of the SQL string itself. This makes TypeORM inherently resistant to SQL injection when used correctly.

STAGE 1 — SAFE: REPOSITORY FIND

`src/routes/users.ts`

```
import { Router, Request, Response } from 'express';
import { AppDataSource } from '../data-source';
import { User } from '../entities/User';

const router = Router();
const userRepo = AppDataSource.getRepository(User);

router.get('/search', async (req: Request, res: Response) => {
  const { name } = req.query;

  // TypeORM repository – name is passed as a parameter, never interpolated into SQL
  const users = await userRepo.find({
    where: { name: String(name) },
  });

  res.json(users);
});

export default router;
```

Output:

```
GET /users/search?name=Alice
→ SELECT * FROM "user" WHERE "name" = $1 -- $1 = 'Alice' (safe parameterised query)

GET /users/search?name=' OR '1'='1
→ SELECT * FROM "user" WHERE "name" = $1 -- $1 = '' OR '1'='1' (literal string, no injection)
```

STAGE 2 — SAFE: QUERYBUILDER WITH PARAMETERS

`src/routes/users.ts (QueryBuilder variant)`

```
router.get('/search-qb', async (req: Request, res: Response) => {
  const { name } = req.query;

  const users = await userRepo
    .createQueryBuilder('user')
    .where('user.name ILIKE :name', { name: `%${String(name)}%` })
    .getMany();

  res.json(users);
});
```

Output:

```
GET /users/search-qb?name=ali
→ SELECT * FROM "user" WHERE user.name ILIKE $1 -- $1 = '%ali%'
```

Warning: TypeORM's raw-query escape hatch — `query(sql, params)` or `QueryBuilder's .andWhere(\`user.name = '${name}'\`)` with template literals — bypasses parameterisation entirely. Avoid string interpolation inside raw SQL fragments. If you must use raw SQL, always pass values via the parameters array: `.andWhere('user.name = :name', { name })`.

6.4 Sanitising Rich Text (Allowing Some HTML)

Some applications genuinely need to store user-generated HTML — comment systems with bold/italic formatting, rich text editors. Escaping everything is too aggressive here; instead, use a sanitiser that removes dangerous tags while preserving safe ones.

```
npm install sanitize-html npm install --save-dev @types/sanitize-html
```

```
src/middleware/sanitise.ts (extended)
```

```
import he from 'he';
import sanitizeHtml from 'sanitize-html';

export function escapeHtml(raw: string): string {
  return he.encode(raw, { useNamedReferences: false });
}

export function sanitiseUserText(raw: unknown): string {
```

```

    if (typeof raw !== 'string') throw new TypeError('Expected a string value');
    return escapeHtml(raw.trim());
  }

  const ALLOWED_TAGS = ['b', 'i', 'em', 'strong', 'a', 'ul', 'ol', 'li', 'p', 'br'];
  const ALLOWED_ATTRS: sanitizeHtml.IOptions['allowedAttributes'] = {
    a: ['href', 'title'],
  };

  /**
   * Strips all HTML except a curated allowlist.
   * Use when storing rich text from a WYSIWYG editor.
   */
  export function sanitiseRichText(raw: unknown): string {
    if (typeof raw !== 'string') throw new TypeError('Expected a string value');
    return sanitizeHtml(raw.trim(), {
      allowedTags: ALLOWED_TAGS,
      allowedAttributes: ALLOWED_ATTRS,
      allowedSchemes: ['https', 'mailto'], // Prevent javascript: hrefs
    });
  }

```

Output:

Input: '

Hello world</p><script>alert(1)</script>click'

Output: '

Hello world</p><a>click'

↑ <script> removed entirely, javascript: href stripped

7. Composing the Controls in a Single Application

Each security control in isolation provides partial protection. This section shows how the three layers compose into a coherent application middleware stack.

src/app.ts (full composition)

```

import express, { Application, Request, Response, NextFunction } from 'express';
import helmet from 'helmet';
import cors, { CorsOptions } from 'cors';
import { globalLimiter, authLimiter } from './middleware/rateLimiter';

```

```
import authRouter    from './routes/auth';
import commentsRouter from './routes/comments';

const ALLOWED_ORIGINS = (process.env.ALLOWED_ORIGINS ?? 'http://localhost:4200')
  .split(',').map(o => o.trim());

const corsOptions: CorsOptions = {
  origin: (origin, cb) => {
    if (!origin || ALLOWED_ORIGINS.includes(origin)) cb(null, true);
    else cb(new Error(`CORS blocked: ${origin}`));
  },
  methods:      ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials:  true,
  maxAge:       86400,
};

const app: Application = express();

// — Layer 1: Security headers —————
app.use(helmet());

// — Layer 2: CORS —————
app.use(cors(corsOptions));
app.options('*', cors(corsOptions));

// — Layer 3: Body parsing (before rate limiting to avoid overhead) —
app.use(express.json({ limit: '10kb' })); // Reject oversized payloads

// — Layer 4: Rate limiting —————
app.use(globalLimiter);

// — Routes —————
app.use('/auth',    authRouter);    // Has its own authLimiter per route
app.use('/comments', commentsRouter); // Uses escapeHtml / sanitiseRichText

// — Centralised error handler —————
app.use((err: Error, _req: Request, res: Response, _next: NextFunction) => {
  console.error(err.message);
  const status = (err as any).status ?? 500;
  res.status(status).json({ error: err.message });
});

export default app;
```

Output:

Request flow:

```

Browser → [Helmet adds response headers]
        → [CORS checks Origin, adds Access-Control-* headers]
        → [body parser reads up to 10 kb]
        → [globallimiter checks IP counter]
        → Route handler
        → [escapeHtml / sanitiseRichText applied before render/storage]
        → Response
  
```

Key Insight: The order of `app.use()` calls is significant. Helmet and CORS must be registered before routes so that every response — including error responses — carries the correct headers. Rate limiting should be registered after body parsing (so the body is readable for logs) but before route handlers (so they are never reached for rejected requests).

Summary

Concept	Key Point
Helmet	Sets defensive HTTP response headers (CSP, HSTS, X-Frame-Options, etc.) that instruct browsers to enforce security policies. One <code>app.use(helmet())</code> call activates sensible defaults; individual headers are configurable.
Content Security Policy	Restricts which origins can load scripts, styles, and media. <code>'unsafe-inline'</code> and <code>'unsafe-eval'</code> in <code>scriptSrc</code> negate most XSS protection. Set <code>frameAncestors: ['none']</code> for APIs that are never embedded.
CORS	

Concept	Key Point
	Browser-enforced; not a server firewall. Use an origin allowlist in production. Set <code>credentials: true</code> when your API uses cookies or the Authorization header — this prohibits wildcards.
express-rate-limit	Counts requests per IP over a sliding window. Apply a global limiter site-wide and tighter limiters on sensitive endpoints (login, password reset). Use Redis store in multi-process deployments.
Input escaping (he)	Converts HTML special characters to entities before rendering into HTML. Prevents XSS from stored user content. Distinct from validation — escaping applies at output time, not input time.
SQL injection and TypeORM	Repository methods and <code>.where(':param', { param })</code> syntax are safe by default. String interpolation inside raw SQL fragments bypasses parameterisation and introduces injection risk.
Rich text sanitisation (sanitize-html)	When users submit HTML (WYSIWYG editors), use a tag/attribute allowlist rather than escaping everything. Block <code>javascript:</code> URI schemes explicitly.
Middleware order	Helmet and CORS first, then body parser, then rate limiting, then routes. Error-handling middleware (four arguments) registered last.

Interview Questions

1. What does the `Content-Security-Policy` header do, and why does Helmet set `x-xss-Protection: 0` instead of enabling it?

Expected answer: CSP instructs the browser to block scripts, styles, and other resources that do not come from explicitly approved origins or match a nonce/hash. `X-XSS-Protection` was an IE-era filter with exploitable bugs — disabling it prevents those bugs from being weaponised; CSP is the modern, standards-based replacement.

2. You have an API that uses session cookies for authentication. A colleague suggests setting `origin: '*'` in your CORS configuration. What is the problem?

Expected answer: Browsers refuse to include cookies (or the `Authorization` header) in requests to a wildcard CORS endpoint. Setting `credentials: true` and `origin: '*'` simultaneously causes the browser to block the response. More importantly, a wildcard origin policy would allow any page on the internet to trigger authenticated requests on behalf of a logged-in user, defeating the same-origin protection that session cookies rely on.

3. Your Express application runs as four Docker containers behind a load balancer. You have applied `express-rate-limit` with the default in-memory store and a limit of 10 requests per minute. An attacker sends 9 requests per minute — one to each container. What is the effective limit and how do you fix it?

Expected answer: The effective limit is $10 \times 4 = 40$ requests per minute, because each container maintains its own counter. The fix is to replace the default store with a Redis-backed store (`rate-limit-redis`) so all instances share a single counter per IP key.

4. Explain the difference between input validation and output escaping. Give an example where you need both.

Expected answer: Validation checks that input conforms to an expected schema (type, length, format) and rejects data that does not. Escaping transforms special characters in output so they cannot be interpreted as code in the target context. A comment system illustrates both: validation

rejects comments that are longer than 2,000 characters or contain no text; output escaping converts any `<script>` tag in an accepted comment to `<script>` before rendering it in HTML, preventing XSS.

5. When using TypeORM's QueryBuilder, how do you safely pass user-supplied values into a WHERE clause?

Expected answer: Pass values via the named-parameter syntax: `.where('entity.column = :value', { value: userInput })`. TypeORM sends the value as a bound parameter to the database driver, which never interpolates it into the SQL string. String template literals inside the SQL fragment (e.g., `.where(`entity.column = '${userInput}'`)`) bypass this mechanism and introduce SQL injection.

6. What does `skipSuccessfulRequests: true` do on an auth rate limiter, and when is it appropriate?

Expected answer: It means that only failed requests (4xx/5xx responses) are counted towards the limit. A successful login does not consume one of the allowed attempts. This is appropriate for login endpoints because legitimate users who occasionally mistype their password should not have their successful logins penalised, while attackers whose attempts continuously fail will still hit the limit.

7. Your application renders user-generated content in HTML templates. A security review flags the use of `sanitize-html` with an empty `allowedTags` array instead of `he.encode()` for plain text fields. What is the reviewer's concern?

Expected answer: An empty `allowedTags` array in `sanitize-html` strips all HTML tags, but it does so by parsing the HTML first and removing elements. The parsed text nodes may still contain decoded entities. For plain text fields where no HTML is expected, `he.encode()` (which escapes the string as-is without parsing it as HTML first) is simpler, more predictable, and less likely to contain parsing edge cases.

8. Why should `express.json({ limit: '10kb' })` be set, and what type of attack does it mitigate?

Expected answer: Without a body size limit, an attacker can send an extremely large JSON payload,

forcing the server to buffer and parse megabytes of data for every request. This consumes memory and CPU, potentially causing a denial-of-service condition. Setting a small limit (10 kb is suitable for most API payloads) means oversized requests are rejected before the body is fully read.

Practical Assignment — Hardening a Comment API

Objective: Apply Helmet, CORS, rate limiting, and output escaping to a small Express + TypeORM API that stores and retrieves user comments. By the end, the application should resist basic XSS, have a defensive header profile, enforce CORS, and throttle abusive clients.

Estimated time: 45–60 minutes

1. **Scaffold the project.** Initialise a TypeScript project with Express and TypeORM connected to SQLite. Create a `Comment` entity with `id`, `author`, and `body` columns (all strings, `body` max length 500).
2. **Add Helmet.** Register `helmet()` as the first middleware. Add a custom CSP directive that allows no external scripts and disallows framing entirely (`frameAncestors: [ "none" ]`). Verify the headers with `curl -I http://localhost:3000/comments`.
3. **Configure CORS.** Allow only `http://localhost:4200` as an origin. Enable `credentials: true`. Register the preflight handler with `app.options('*', cors(corsOptions))`. Test that a request from `http://evil.example.com` is rejected with 403.
4. **Add rate limiters.** Apply a global limiter of 60 requests per minute to all routes. Create a separate `POST /comments` limiter of 5 submissions per minute. Use `standardHeaders: true` so clients can see their remaining quota.
5. **Implement the POST /comments route.** Accept `author` and `body` from `req.body`. Run both through `sanitiseUserText()` before saving to the database. Return the saved entity as JSON.
6. **Implement the GET /comments route.** Retrieve all comments from TypeORM. Return them as JSON. (JSON responses do not require HTML escaping — the browser treats JSON as data, not markup. No additional escaping is needed here.)

7. Add a `GET /comments/html` route that renders comments into an HTML string using template literals. Apply `escapeHtml()` to each `author` and `body` before interpolation. Verify by posting a comment with body `<script>alert(1)</script>` and checking the rendered HTML.
8. Verify the complete header set. Run `curl -I http://localhost:3000/comments` and confirm the presence of `Content-Security-Policy`, `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `RateLimit-Limit`, and `Access-Control-Allow-Origin`.

Output:

```
POST /comments { "author": "<b>Alice</b>", "body": "<script>alert(1)</script> Nice post!" }
→ 201 Created
  { "id": 1, "author": "&lt;b&gt;Alice&lt;/b&gt;", "body": "&lt;script&gt;alert(1)&lt;/script&gt; Nice post!" }

GET /comments/html
→ 200 OK (Content-Type: text/html)
<li><strong>&lt;b&gt;Alice&lt;/b&gt;</strong>: &lt;script&gt;alert(1)&lt;/script&gt; Nice post!</li>
↑ Script tag rendered as visible text – no execution
```

Bonus Challenge: Extend the rate limiter to key on `req.ip` but also on a custom `X-Device-ID` header when present, so that a single device is limited even if it rotates IP addresses. Implement a custom `keyGenerator` function in the rate limiter options and write a short test using `supertest` that fires six requests with the same header and confirms the sixth receives a 429.

Note — Additions and Changes to Syllabus Coverage:

- **Added:** A dedicated section (6.1) distinguishing validation, escaping, and sanitisation — the syllabus lists "Input escaping" as a bullet without elaborating on this distinction, which is a common source of confusion.
- **Added:** Coverage of `sanitize-html` for rich-text use cases, since the syllabus only mentions "input escaping" generically. Applications with WYSIWYG editors require a different approach than plain-text escaping.

- **Added:** The Redis-backed rate limiter stage, as the syllabus does not address multi-process deployments explicitly but this is a practical production concern that students building real APIs will encounter.
- **Added:** `express.json({ limit: '10kb' })` note. Body size limits are not in the syllabus for this section but are a zero-configuration quick win that pairs naturally with rate limiting.
- **TypeScript:** All code uses TypeScript as per the instruction added 1 March 2026.

